

unit → 1

→ Digital Computers

- A digital computer is a digital system that performs various computational tasks using variables that take a limited number of discrete values.
- They function more reliably, if two states i.e. binary inputs (0/1) are used.

→ Hardware

- It consists of the electronic components and electromechanical devices that comprise of the physical entity of the device.

→ Software

- It consists of instructions and data that the computer manipulates to perform various data processing tasks.

→ Need to study COA?

- To understand how a computer executes a program internally and helps to write more effective programs.
- It helps to understand the reason behind poor execution of efficient algorithm.

→ Computer Architecture

- It's concerned with structure & behaviour of computer.
- It defines what hardware components a computer needs and how they connect and

communicate with each other, so computer can run software & execute commands

→ Computer Organization
concerned with the way hardware components operate & how they are connected together

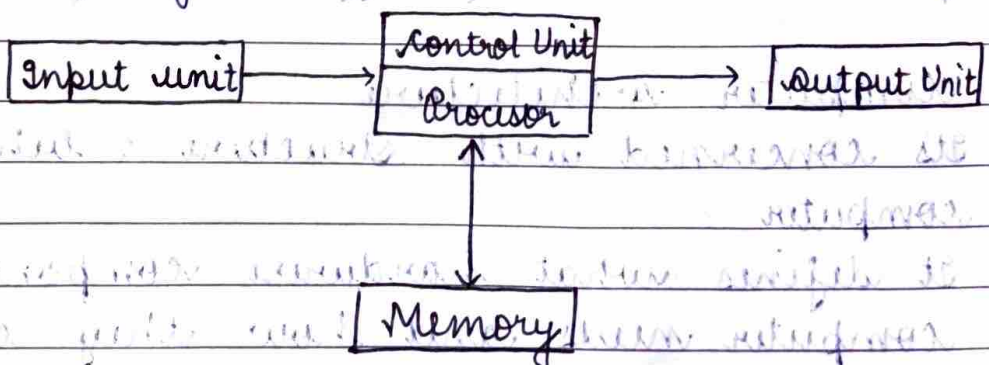
- It deals with how different components of a computer are interconnected and how they operate during execution
- Organisation implements the architecture

→ Computer Design

- Organisation is achieved via computer design
- It's concerned with hardware design of computer
- It determines what hardware is to be used and how the hardware are to be connected

→ von Neumann Computers

- It's the design blueprint behind almost every modern computer
- It stored both data and programs (instruction) together in same memory space
- During execution of program, stored instructions were fetched from memory and were decoded to set up necessary data paths and generate control signals

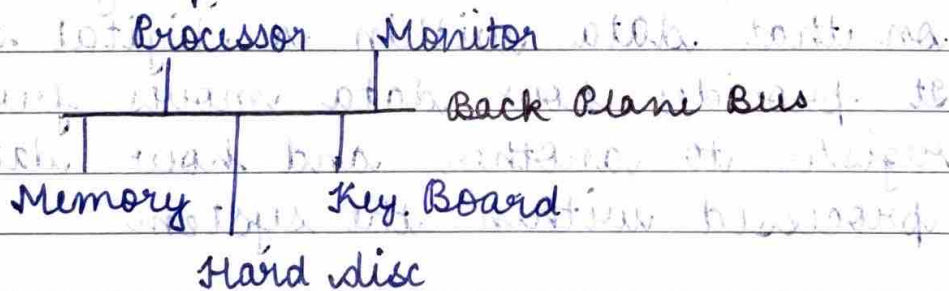


→ Shortcomings of von Neumann computers

- Only one single connection exists between the processor and memory so, consequently at any time only one memory can be accessed. It means, at a time either an instruction can be fetched or a data item can be accessed.
- It means there was a problem in parallel execution of instructions and this was the "bottleneck" of von Neumann computers in achieving high performance computations.

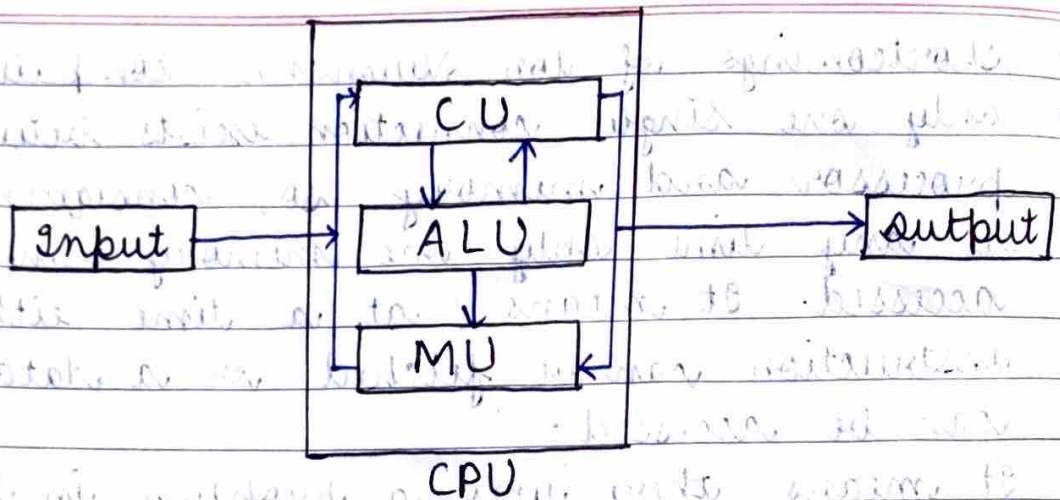
→ Basic organisation of a computer

- In simplest organisation of computers, all the parts can be connected through a single bus called backplane bus.



1. Processor

- Fetches an instruction stored in memory and executes it.
- Contains arithmetic & logic units to manipulate data, registers to store data & control unit to generate control signals for fetching, executing & storing.



Block diagram of computer

→ Register transfer language

- It's a symbolic notation used in digital logic design to describe how a data moves between registers and how operations like addition, subtraction & shifting are performed on that data within a digital system.
- It provides how data moves from one register to another and how data is processed within the system.

→ Micro operations

- Fundamental low level operations performed on data stored in registers such as arithmetic (+, -), logical (AND, OR) or shifting.

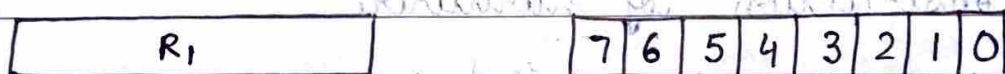
→ Registers

- They are groups of flip flops that store binary information forming the basic storage units within a digital system.

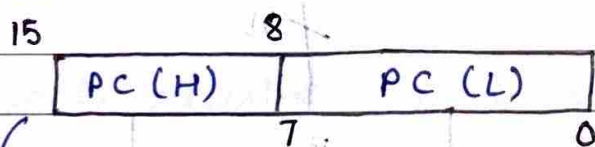
- internal hardware organisation is best defined by
1. specifying the set of registers it contains and their functions
 2. sequence of micro operations performed on binary information stored in registers
 3. control that initiates sequence of micro operations

→ Register Transfer

- Computer registers are designated by capital letters to denote function of register
- eg → Register storing memory is called memory address register & is denoted by MAR, PC for program counter, IR for instruction register & R1 for processor register

Register R_1

← right to left numb.
8 bit register



→ 16 bit register divided into 2 parts high & low

PC(0-7) or PC(L) refers to low order byte
PC(8-15) or PC(H) refers to high order byte

$R_2 \leftarrow R_1$ contents of reg R_1 into reg 2

↳ replacement operator

→ Now R_2 has contents of R_1 , but R_1 also & will always have contents that it had

- For transfer to occur in a predetermined condition :-

if $(P=1)$ then $(R_2 \leftarrow R_1)$



control signal generated in control section

- We can also use control function :-

- A control function is a boolean expression that's equal to 0/1

$P : R_2 \leftarrow R_1$



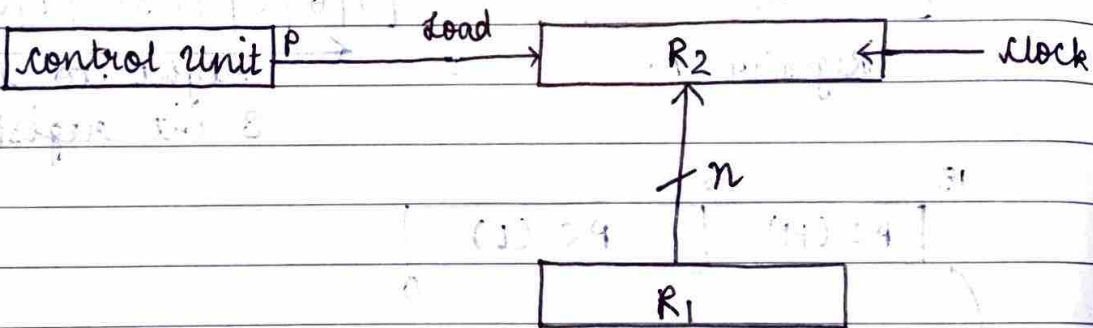
terminates control function

$R_2 \leftarrow R_1$ occur

only when P is true

i.e. $P=1$

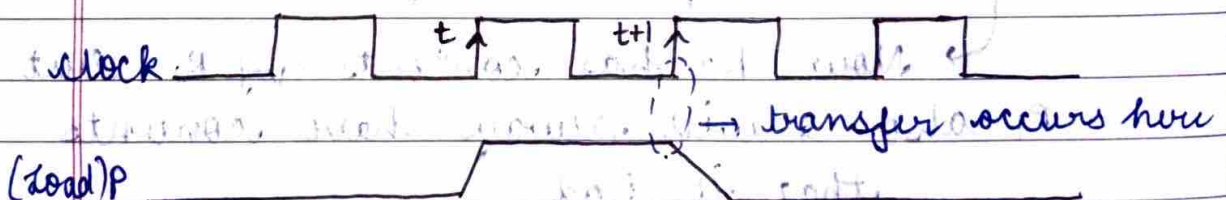
- every statement of RTL is implemented by construction of hardware



no. of bits of register

- n outputs of R_1 are connected to n inputs of R_2 . R_2 has a load input that's activated by P .

- if P becomes active at time ' t ', the register is triggered at next +ve transition of clock at time ' $t+1$ '



T: $R_2 \leftarrow R_1$ $R_1 \leftarrow R_2$

① → separates two micro operations that are being performed during one common clock.

→ Bus Transfer

- A bus structure consists of a set of common lines, one for each bit of a register, through which binary information is transferred one at a time.
- Control signals are used to determine which register is selected by bus during each particular register transfer.
- One common way to construct a bus system is using multiplexers; the multiplexer selects the source register whose memory is placed in the bus.

→ Bus system for 4 registers

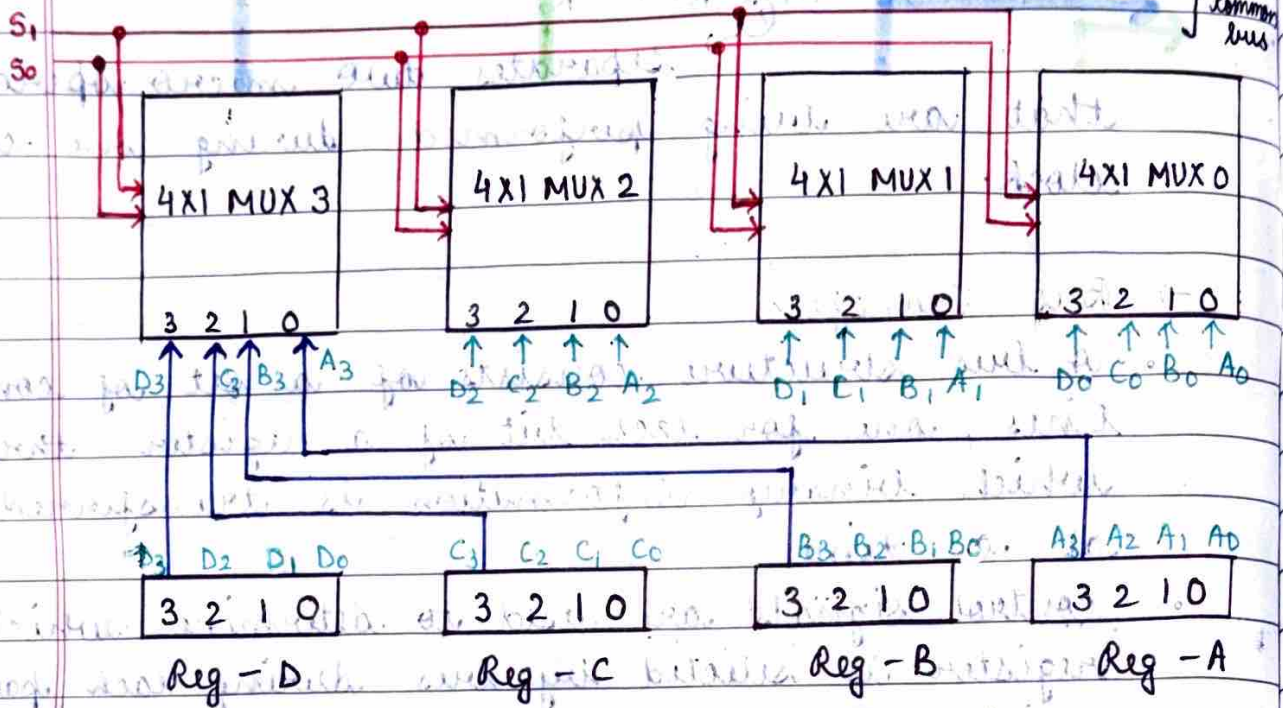
- Each register has 4 bits (0-3) & bus consists of four 4x1 multiplexers having 4 data inputs & 2 selection inputs.
- One multiplexer will handle only one type of bit eg → M₀ handles 0th bit of all the registers.
- Two select lines S_1 & S_0 .
 S_1 & S_0 both = 0 → then MUX chooses 0th inputs of all like A_0, B_0, C_0, D_0 .

$\bar{S}_1 \bar{S}_0 \rightarrow A_0, A_1, A_2, A_3 \rightarrow A$

$\bar{S}_1 S_0 \rightarrow B_0, B_1, B_2, B_3 \rightarrow B$

$S_1 \bar{S}_0 \rightarrow C$

$S_1 S_0 \rightarrow D$



Bus transfer in RTL is depicted by:-

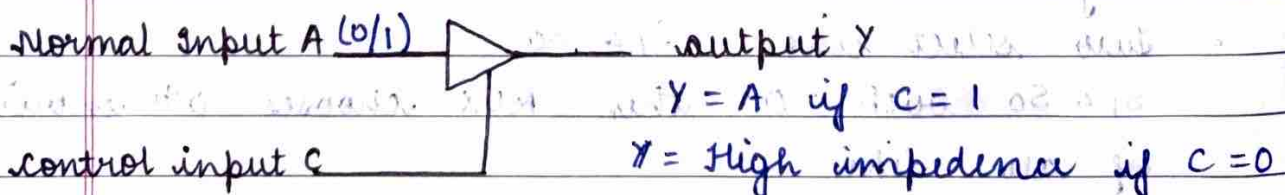
$$BUS \leftarrow C, R_1 \leftarrow BUS$$

Data of x enters bus & is then sent to R_1

→ Three State Buffer

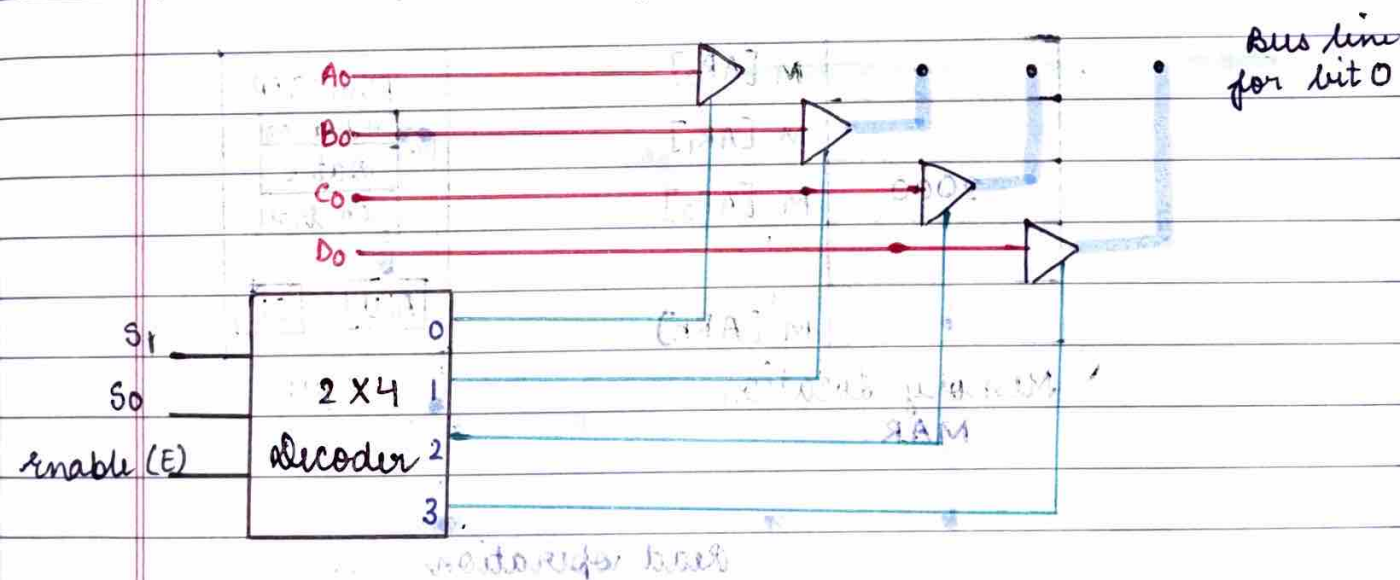
- A bus can also be constructed using three state gate which exhibits three states, namely 0, 1 and high impedance state (which behaves like an open circuit)

Most commonly used three state gate in a bus is a buffer



3 state Buffer

- In bus, 4 buffers are connected together to form single bus line and control inputs determine which of the 4 normal inputs will communicate with bus line.
- Not more than one buffer state can be active at a time and only one buffer can be in active state at a given time. At a time only 1 buffer can be in active state & rest must be in high impedance state.
- A decoder is used for this purpose.
- When enable input of decoder is 0, all 4 outputs are zero & bus line is in high imped. state.
- When enable input is active, one of the four buffers will be active depending upon binary value in select inputs of decoder.

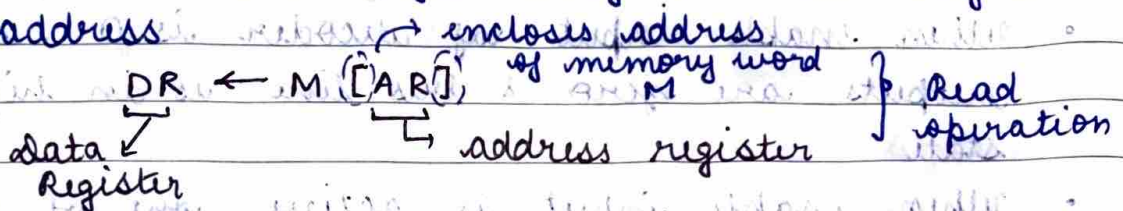


For making a four register of n bit each bus we need n circuits with four buffers in each & each group of 4 buffers receives one significant bit from the four registers & each common output produces one of the lines

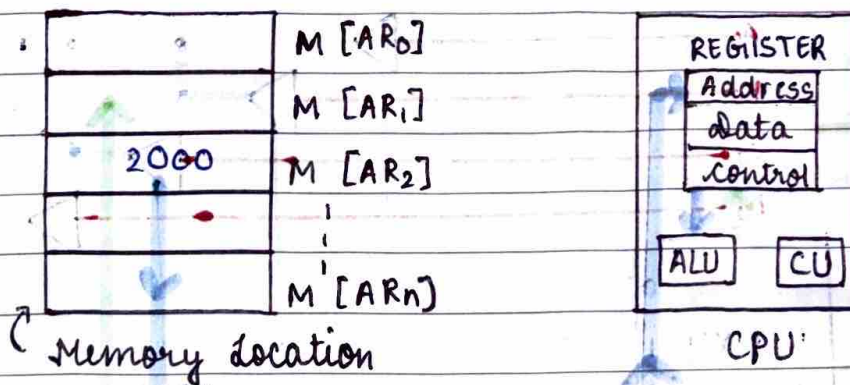
for common bus for total n lines.
only one decoder is necessary to select between 4 registers.

→ Memory Transfer

- Transfer of information from a memory word to outside environment is called read operation & transfer of new information in memory is called write operation.
- A memory word is represented by 'M' & is selected amongst many available by its address.



$M[\text{AR}] \leftarrow R$ → Write operation



Read operation

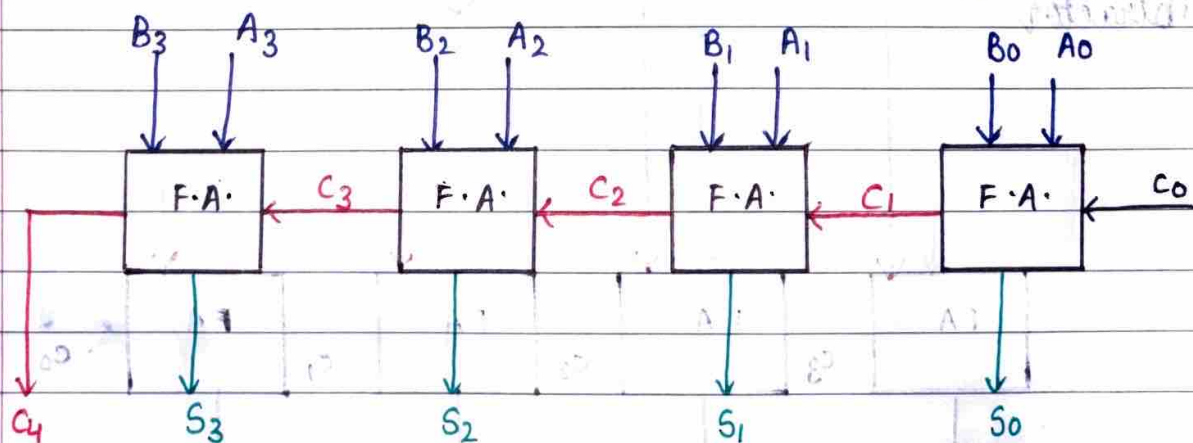
Write operation

→ Arithmetic Micro operations

1. $R_3 \leftarrow R_1 + R_2$
 - Transfers contents of $R_1 + R_2$ into R_3
2. $R_3 \leftarrow R_1 - R_2$ ($R_3 \leftarrow R_1 + \overline{R_2} + 1$)
 - Contents of $R_1 - R_2$ transferred to R_3
 - Performed by adding 2's complement of R_2 in R_1
3. $R_2 \leftarrow \overline{R_2}$
 - complements contents of R_2 (1's complement)
4. $R_2 \leftarrow \overline{R_2} + 1$
 - 2's complement of R_2 (negates)
5. $R_1 \leftarrow R_1 + 1$
 - Increments contents of R_1 by 1
6. $R_1 \leftarrow R_1 - 1$
 - Decrements the contents of R_1 by 1

→ Binary Adder

- A binary adder generates the arithmetic sum of binary numbers of any length and can be constructed by cascading full adder circuits.
- Output carry of one full adder is connected to input carry of next full adder.
- n bit binary adder requires n full adders.



4 bit binary adder

$C_4 = 1$ → $C_4 = 0$
-ve +ve

represent
in 2's
complement

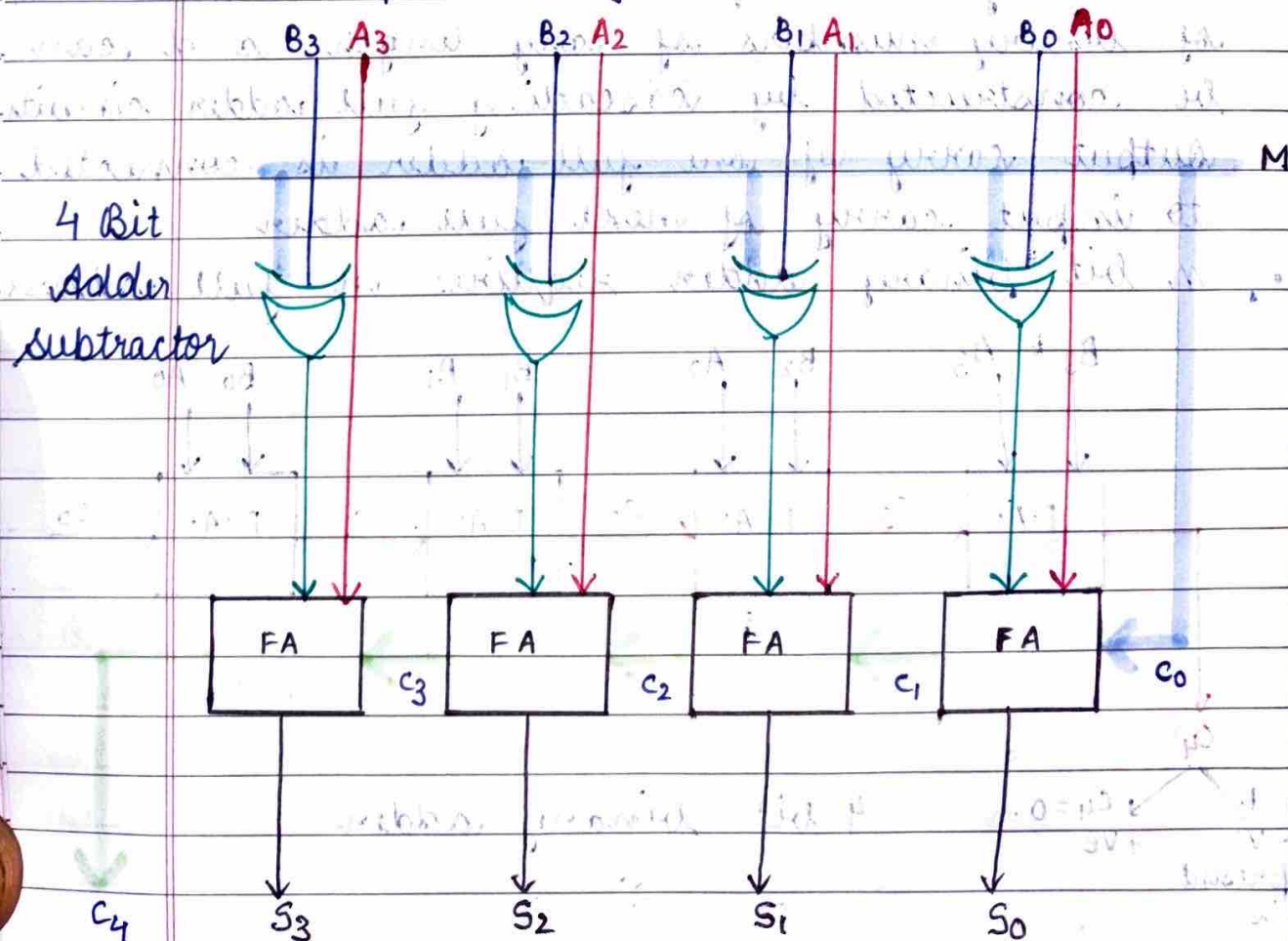
→ Binary Adder Subtractor

- Subtraction can be done by using 2's complement of B & adding that to A
 - Addition & subtraction can be performed in one common circuit by including an XOR gate with each full adder
 - Mode input M controls the operation, when $M=0$, circuit is adder & for $M=1$, circuit becomes subtractor
 - Each XOR gate receives M & B_i as input
1. $M=0$ $M \oplus B = 0 \oplus B = +B$ & $C_{in}=0$
 ↳ performs $A+B$

2. $M=1$ $M \oplus B = 1 \oplus B = \bar{B}$ & $C_{in}=1$

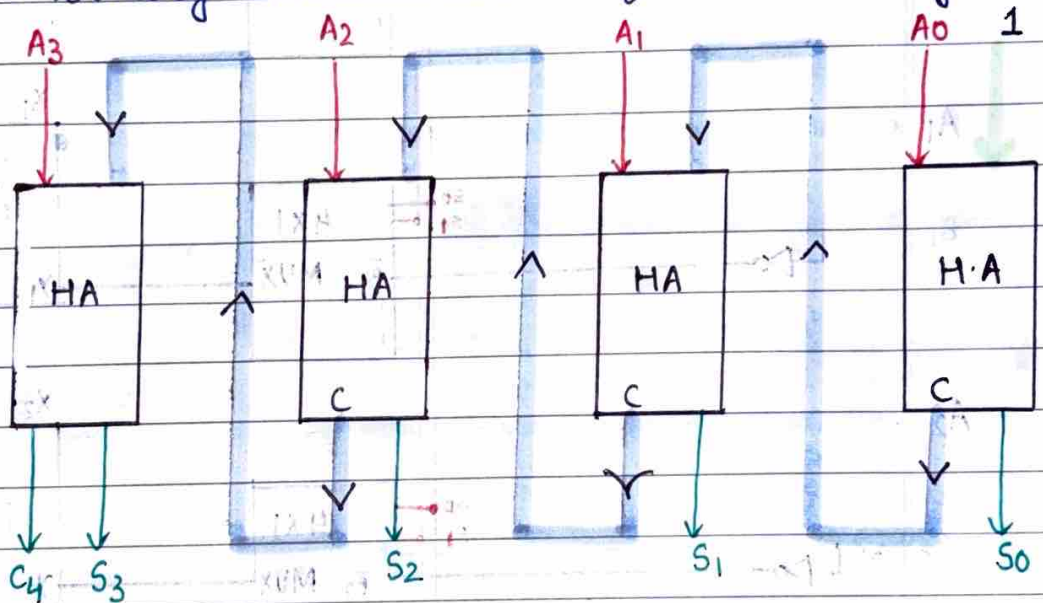
↳ performs $A + (2's \text{ complement of } B)$

> For unsigned nos it gives $A+B$ if $A \geq B$ or 2's complement of $(B-A)$ if $A < B$



→ Binary Incrementor

- It adds one to the number in a register
eg $\rightarrow 0110$ goes to 0111 after incrementation
- It's implemented using binary counter
- Every time the count enable is active, the clock pulse transition increments the content of register by 1
- It can also be implemented by cascading the half adders
- n bit binary incrementor requires n half adders



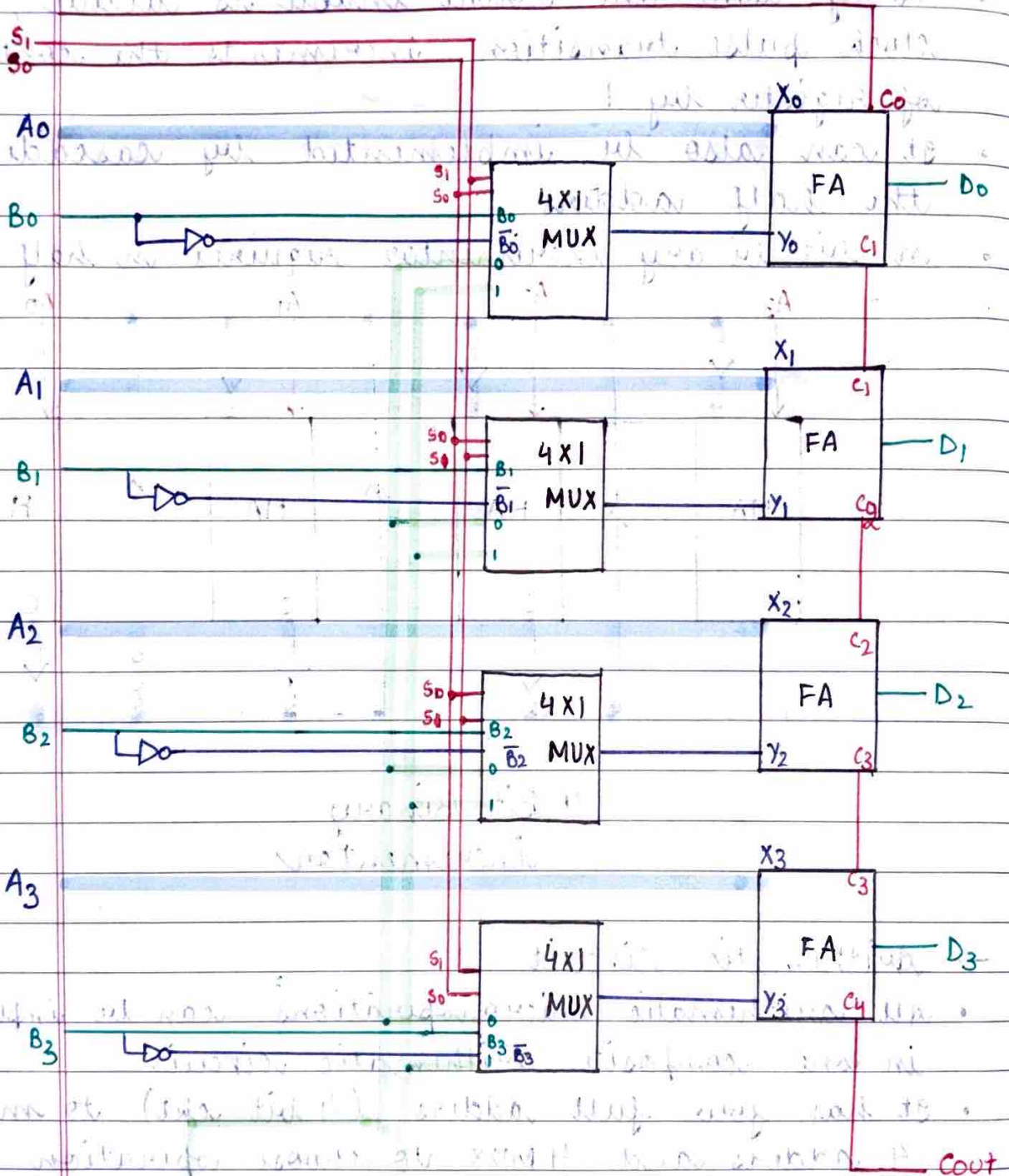
4 Bit Binary
incrementor

→ Arithmetic circuit

- All arithmetic micro operations can be implemented in one composite arithmetic circuit
- It has four full adders (4 bit ckt) to make 4 adders and 4 MUX to choose operation
- If we have 2 inputs A & B (of 4 bits) and output D so, all bits of A go into $F \cdot A$ while of B in MUX along with $\bar{B}$, 0 & 1

- The output of the binary adder is calculated from $D = A + Y + C_{in} \rightarrow$ input carry (0/1)

4 bit binary
no at X inputs
4 bit binary
at Y inputs



4 Bit arithmetic circuit

→ Arithmetic circuit function table

S_1	S_0	C_{in}	Y	$D = A + Y + C_{in}$	MICRO OPERATION
0	0	0	B	$D = A + B$	Add
0	0	1	B	$D = A + B + 1$	Add with carry
0	1	0	$\bar{B}$	$D = A + \bar{B}$	Subt. with borrow
0	1	1	$\bar{B}$	$D = A + \bar{B} + 1$	Subtract
1	0	0	0	$D = A$	Transfer A
1	0	1	0	$D = A + 1$	Increment A
1	1	0	1	$D = A - 1$	Decrement A
1	1	1	1	$D = A$	Transfer A

→ Logic Micro operations

- Logic micro operations specify binary operations for string of bits stored in registers.
- They treat each bit of the variable separately.

eg → $P: R_1 \leftarrow R_1 \oplus R_2$

if $R_1 = 1010$ & $R_2 = 1100$

$$\begin{array}{r} 1010 \\ \oplus 1100 \\ \hline 0110 \end{array}$$

(XOR is high for odd

no of 1's)

after P is true, 0110 becomes R_1 's content

• Symbols of logic micro operations

$\vee \rightarrow$ OR

$\wedge \rightarrow$ AND

$\sim$ or $\bar{A} \rightarrow$ NOT

$P + Q$

OR between two boolean expressions

$R_1 \leftarrow R_2 + R_3, R_4 \leftarrow R_5 \vee R_6$

Arithmetic Addition

OR

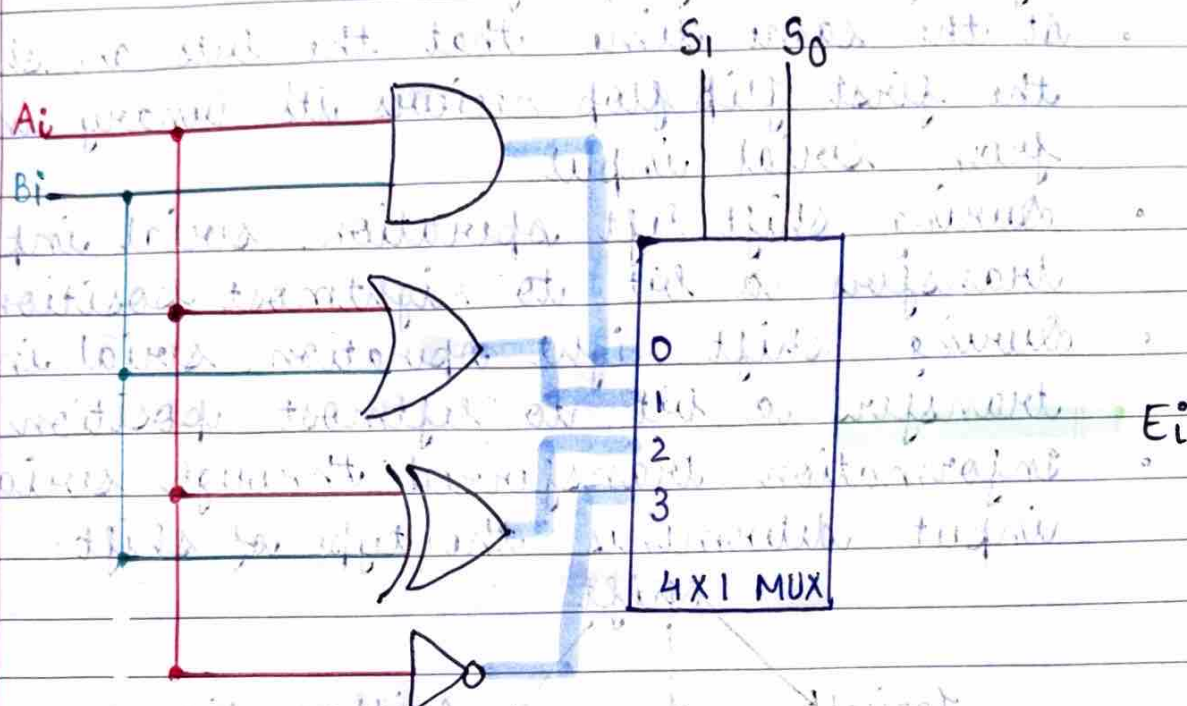
- List of logic micro operations
- 16 logical operations can be performed by using 2 binary variables

Boolean function	Micro operation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	And
$F_2 = xy'$	$F \leftarrow A \wedge \bar{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'y$	$F \leftarrow \bar{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	exclusive OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x + y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	exclusive NOR
$F_{10} = y'$	$F \leftarrow \bar{B}$	Complement B
$F_{11} = x' + y'$	$F \leftarrow \overline{A \vee B}$	
$F_{12} = x'$	$F \leftarrow \bar{A}$	complement A
$F_{13} = x' + y$	$F \leftarrow \bar{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow 1$	Set all to 1

In this table x belongs to bits of binary content of A and y belongs to bits of binary contents of B

- Most computers only perform OR, AND, NOT & XOR
- Outputs of 4 gates are applied to MUX to choose operation
- For a logic circuit with n bits, this

Diagram should be repeated in times
(one time for each bit)

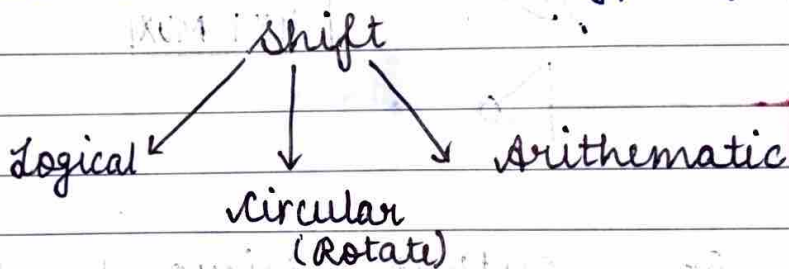


S_1	S_0	output	Micro operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	NOT

→ Applications of logical micro operations

- Used to manipulate individual bits
- Used to change bit values, delete group of bits & insert new bits to register
- It can be used to perform selective set operation, selective complement, selective clear, mask operation (AND), insert operation (OR), clear operation (XOR)

- shift micro operations
- shift micro operations are used for serial transfer of data to shift the contents of register left / right.
 - At the same time that the bits are shifted the first flip flop receives its binary info from serial input
 - During shift left operation, serial input transfers a bit to rightmost position
 - During shift right operation, serial input transfers a bit to leftmost position
 - Information transferred through serial input determines the type of shift.



1- Logical Shift

$R_1 \leftarrow \text{shl } R_1$ (Logical shift left)

$R_1 \leftarrow \text{shr } R_1$ (Logical shift right)

Let's take eg of 1011

$\text{discarded} \leftarrow \text{1} \quad 0 \quad 1 \quad 1$
 $\text{0} \quad \text{1} \quad \text{1} \quad \text{0} \rightarrow \text{zero at rightmost (shift left)}$

$1011 \xrightarrow{\text{shl}} 0110$

$\text{zero at leftmost} \leftarrow \text{0} \quad \text{0} \quad \text{1} \quad \text{1}$
 $\text{1} \quad \text{0} \quad \text{1} \quad \text{1} \rightarrow \text{discarded}$

(shift right)

$1011 \xrightarrow{\text{shr}} 0101$

if we have to move 2 bits
discarded $\leftarrow 01011001$ she
 01101000

- In logical shifts either leftmost/rightmost bits are discarded (problem of logical shift)

2. circular shift

$R_1 \leftarrow \text{cil } R_1$ (circular left shift)

$R_1 \leftarrow \text{cir } R_1$ (circular right shift)

- Circular shift right

eg $\rightarrow 1011$

Transfers rightmost bit to leftmost end (not discarded)

- Circular shift left

eg $\rightarrow 1011$

Transfers leftmost bit to rightmost end (not discarded)

$1011 \xrightarrow{\text{cil}} 0111$

$1011 \xrightarrow{\text{cir}} 1110$

3. Arithmetic shift

- Used to perform operations on signed numbers

$R_1 \leftarrow \text{ashl } R_1$ (arithmetic shift left)

$R_1 \leftarrow \text{ashr } R_1$ (arithmetic shift right)

eg $\rightarrow 0010$

$\swarrow \searrow \swarrow \searrow$ } ashl

$\swarrow \searrow \swarrow \searrow$ } ashr

gives product with 2
of original no.
 $0010 \rightarrow 2$ whose ashl
is 0100 i.e. 4

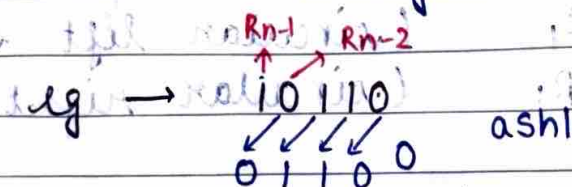
on taking ashr
decimal gets $\div$ by 2
 $0010 \rightarrow 2$ whose
ashr is $0001 \rightarrow 1$

- In arithmetic shift left, if first two leftmost bits aren't same then shift left would lead to change in sign causing overflow so we need to change (reverse) signed bit.

$$V_s = R_{n-1} \oplus R_{n-2}$$

$V_s = 0 \rightarrow$ no overflow

$V_s = 1 \rightarrow$ overflow



10110 $\xrightarrow{\text{ashl}}$ 01100

$\downarrow$ -ve

$\downarrow$ +ve

(overflow)

$\Rightarrow$ ashl

11100

reverse signed bit

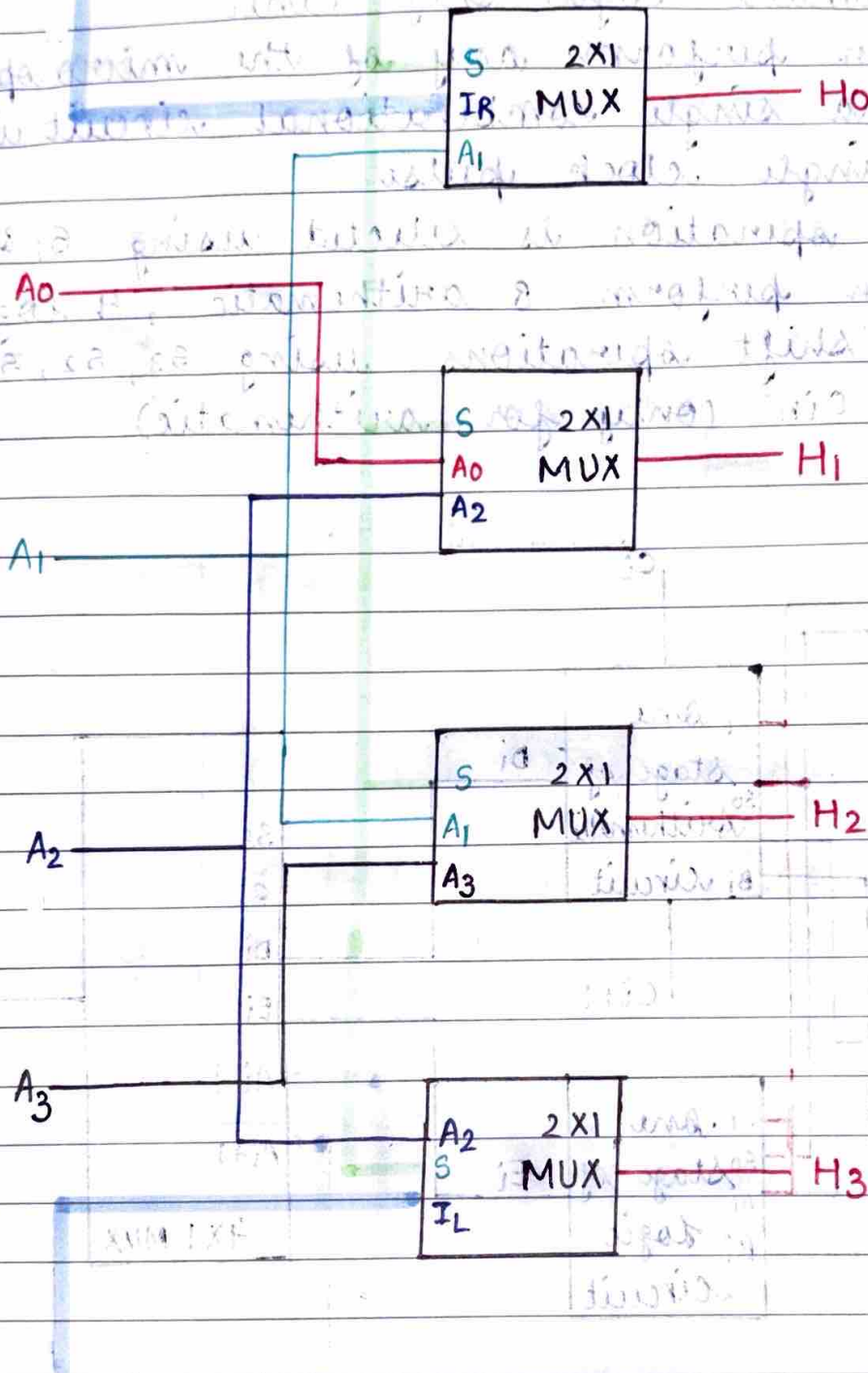
- It's implemented using combinational circuits
- The content of register that has to be shifted is first placed on a common bus whose output is connected to combinational shifter & shifted no is then loaded back to register.
- This requires only one clock pulse for loading shifted value into register.
- 4 bit shifter has 4 inputs ($A_0 - A_3$) 4 outputs ($H_0 - H_3$) with 2 serial inputs (one for left shift & other for right shift I_L & I_R)
- $S=0 \rightarrow$ shifted right
- $S=1 \rightarrow$ shifted left

select input
 0 for shift right
 1 for shift left

classmate

Date _____
 Page _____

serial input (I_R)



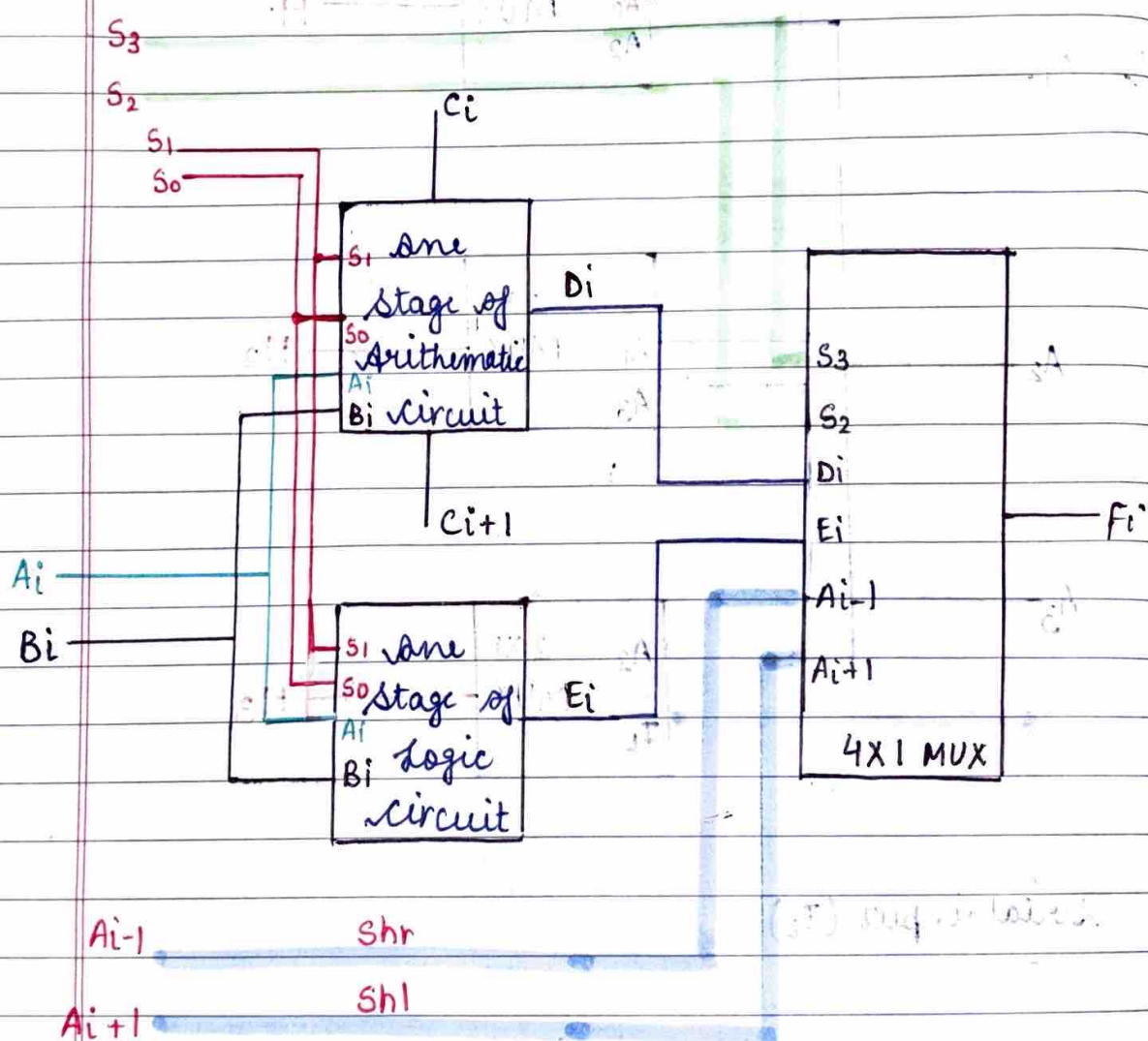
serial input (I_L)

Function Table

S	H_0	H_1	H_2	H_3
0	I_R	A_0	A_1	A_2
1	A_1	A_2	A_3	I_L

→ Arithmetic Logic Shift unit

- It can perform any of the micro operation using a single combinational circuit in a single clock pulse
- Micro operation is selected using S_1 & S_0
- It can perform 8 arithmetic, 4 logic & 2 shift operations using S_3, S_2, S_1, S_0 and C_{in} (only for arithmetic)



→ Function Table for Arithmetic Logic Shift Unit

S_3	S_2	S_1	S_0	C_{in}	OPERATION	FUNCTION
0	0	0	0	0	$F = A$	Transfer A
0	0	0	0	1	$F = A + 1$	Increment A
0	0	0	1	0	$F = A + B$	Addition
0	0	0	1	1	$F = A + B + 1$	Add with carry
0	0	1	0	0	$F = A + \bar{B}$	Sub with borrow
0	0	1	0	1	$F = A + \bar{B} + 1$	Subtract
0	0	1	1	0	$F = A + 1$	Increment A
0	0	1	1	1	$F = A$	Transfer A
0	1	0	0	X	$F = A \cdot B$	AND
0	1	0	1	X	$F = A \vee B$	OR
0	1	1	0	X	$F = A \oplus B$	XOR
0	1	1	1	X	$F = \bar{A}$	NOT
1	0	X	X	X	$F = \text{Shr } A$	Shift right A
1	1	X	X	X	$F = \text{Shl } A$	Shift left A

→ Instruction codes

- A computer instruction is a binary code that specifies a sequence of micro operation for a computer.

Instruction codes are stored along with data in the memory.

The computer reads each instruction from memory & places it in control registers which interprets the binary code of the instruction & executes it.

Instruction code is a group of bits that instructs computer to perform a specific operation.

→ Operation

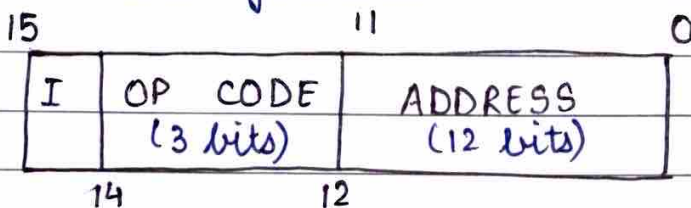
An operation is a part of instructions stored in computer memory.

It's a binary code that tells computer to perform a specific operation.

- For every operation code the control unit starts a sequence of microoperation hence it's also called Macro operation.

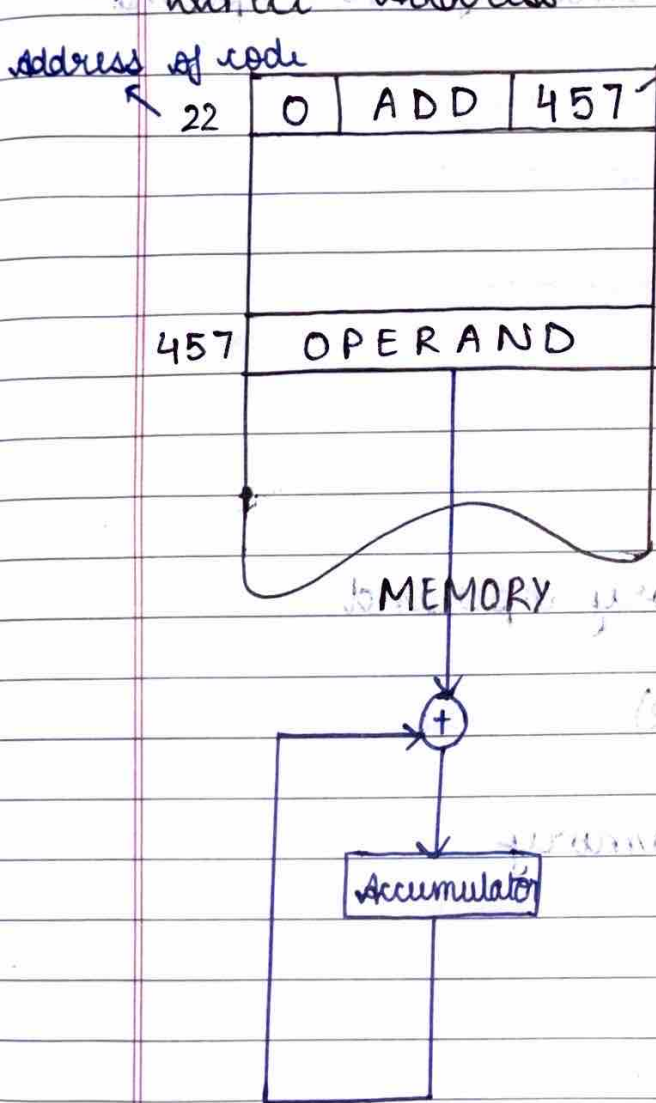
→ Instruction format

- A set of bits specifying the operation to be performed.
- For 16 bit format:-



- Address specifies address of the operand located in main memory
- Address is of 12 bits & can store 2^{12} words of memory
- Op code specifies operation to be performed it has 3 bits and 8 operations (2^3)
- I (Indirect bit) - 000
 - If $I = 0$ (Direct address)
 - If $I = 1$ (Indirect address)

→ Direct Address



EFFECTIVE ADDRESS

If $I = 0$, address specifies operand's address

Address me jo mention hoga, us address pe (jo) jo na hoga memory me wo operand

(IA) assigned with 000000000000

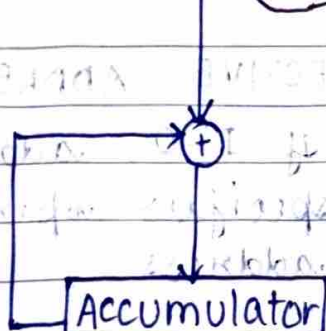
program ref karta hai

(IA) accumulator is register assigned

(IA) assigned with 000000000000

→ Indirect Address

35	1	ADD	300
300	1350		
1350	OPERAND		



If $I=1$, the address mentioned in the instruction contains address of operand.

300 → add. of add. of operand

1350 → operand's address

content of 1350 → operand

→ Computer Registers

1. Data Register (DR)

- Has 16 bits
- It holds the memory operand

2. Address Register (AR)

- 12 bits
- Holds address for memory

3. Accumulator (AC)

- Processor register

4. Instruction Register (IR)

- Holds instruction code

5. Program Counter (PC)
 - Holds address of instruction
 6. Temporary Register (TR)
 - Holds temporary data
 7. Input Register (INPR)
 - Holds input character
 8. Output Register (OUTR)
 - Holds output character
- A basic computer has 8 registers, a memory unit and a control unit
 - A common bus is used to transfer information between registers
 - Outputs of 7 registers & memory are connected to common bus & are selected by S_1, S_2, S_0
 - The register whose load input is enabled (LD) receives data from bus during the next clock transition
 - Memory receives contents of bus when its write input is activated & places the 16 bit data onto bus when read input is activated
 - No along each output shows decimal equivalent of required binary combo
 for eg $DR \rightarrow 3 \Rightarrow 011 = S_2 S_1 S_0$ places data from DR on bus

→ Common Bus System

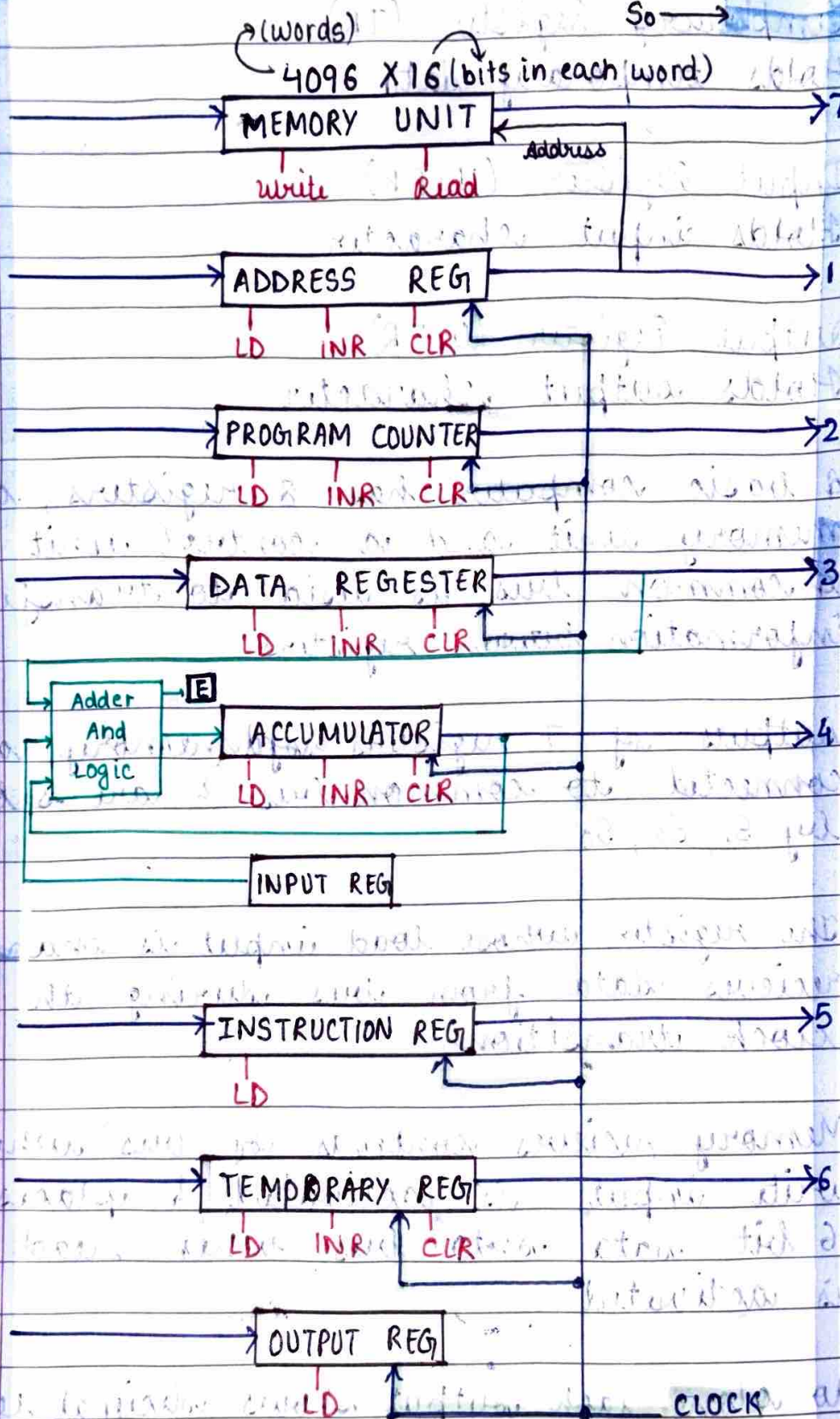
INCREMENT → INR
CLEAR → CLR

LD → LOAD

S₂ → Bus

S₁ →

S₀ →



16 BIT COMMON BUS

→ Computer Instructions

computer instructions

- A computer instruction is a low level binary executed by CPU to perform an operation.
- Instructions serve as bridge between hardware & software.
- It includes opcode, operands & addressing modes.
- It's of three types (All are 16 bit codes)
 1. Memory reference instructions
 2. Register reference instructions →

0111	Reg. operation
------	----------------
 3. Input output instructions

1111 input op

op code = 111

I = 0

→ Memory Reference instructions

I | OP CODE | ADDRESS

11 0 → 000 to 011

12 bits address, 3 bits op code & 1 bit indirect

I = 0 → direct address

I = 1 → indirect address

- Performs operation on an operand located in memory

→ instruction set completeness

The set of instructions is said to be complete if the computer includes a sufficient no of instructions from each of the following category.

1. Arithmetic, logical & shift instructions
2. Instructions for moving to and from memory & processor registers
3. Program control instructions along with instructions that check status conditions
4. Input & output instructions

→ Timing & control

- Timing of all registers in a basic computer is controlled by a master clock generator
- The clock pulses are applied to all the flip flops and registers in the system
- These clock pulses don't change the state of register until it's enabled by control signal which are generated by control unit and provides control inputs for MUXes in common bus

control organisation

Hardwired control

→ control logic implemented using gates, flip flops decoders & other ckts

→ can be optimised to produce fast mode of operation

Microprogramed control

→ control info is stored in control memory which is programmed to initiate the required sequence of microoperations

- Hardwired control requires changes in wiring of various components if design has to be modified
- In microprogrammed control, any required changes can be done by updating the micro program in the control memory

→ Instruction cycle

- Program is a sequence of instructions which is executed in computer by going through a cycle for each instruction.
- An instruction cycle consists of following phases in a basic computer
 1. Fetch an instruction from the memory
 2. Decode the instruction
 3. Read the effective address from memory if the instruction has indirect address
 4. Execute the instruction

1. Fetch & Decode

- Program counter is loaded with address of first instruction of the program
- Sequence counter SC is cleared to 0 providing a decoded timing signal T_0 .
- After each clock cycle, SC is incremented by 1 so we get a sequence of timing signals $T_0, T_1, T_2, T_3, \dots$
- The micro operations for fetch & decode phases can be specified by :-

$T_0 : AR \leftarrow PC$

$T_1 : IR \leftarrow M[AR], PC \leftarrow PC + 1$

$T_2 : D_0, D_1, \dots, D_7 \leftarrow \text{Decode } IR(12-14), AR \leftarrow IR(0-11)$

$I \leftarrow IR(15)$

1. Program counter gave address to address register in T_0 .
2. Instruction in that address is read from memory & placed in instruction register in T_1 & PC is incremented to give the address of next instruction

3. op code in IR is decoded and indirect bit is transferred to flip flop I at T_2 .

4. After the instruction is executed, clock returns to fetch phase with $T_0 = 1$ and SC is cleared to 0.

→ Memory Reference Instructions

There are 7 memory reference instructions for each decoded output from D_0 to D_6 .

Symbol	op. decoder	Symbolic desc.
AND	D_0	$AC \leftarrow AC \wedge M[AR]$
ADD	D_1	$AC \leftarrow AC + M[AR], E \leftarrow \text{Cout}$
LDA	D_2	$AC \leftarrow M[AR]$
STA	D_3	$M[AR] \leftarrow AC$
BUN	D_4	$PC \leftarrow AR$
BSA	D_5	$M[AR] \leftarrow PC, PC \leftarrow AR + 1$
ISZ	D_6	$M[AR] \leftarrow M[AR] + 1$ if $M[AR] + 1 \neq 0$ then $PC \leftarrow PC + 1$

1. AND to AC

- Performs AND logic operation on pairs of bits in AC & memory word is specified by specific address.
- Result is transferred to AC

D_0 to T_4 : $DR \leftarrow M[AR]$

D_0 to T_5 : $AC \leftarrow AC \wedge DR$ $SC \leftarrow 0$

To start
new instruction
cycle
from T_0 .

2. ADD to AC

- Adds the content of memory word specified to the contents of AC.
- Sum is transferred to AC and carry to E (extended accumulator flip flop)

$$D_1 T_4 : DR \leftarrow M[AR]$$

$$D_1 T_5 : AC \leftarrow AC + DR, E \leftarrow Cout, SC \leftarrow 0$$

3. LDA (Load to AC)

- Transfers the memory word specified in instruction to AC

$$D_2 T_4 : DR \leftarrow M[AR]$$

$$D_2 T_5 : AC \leftarrow DR, SC \leftarrow 0$$

4. STA (Store AC)

- Stores the contents of AC into memory word specified.

$$D_3 T_4 : M[AR] \leftarrow AC, SC \leftarrow 0$$

5. BUN (Branch Unconditionally)

- Transfers the program to the instruction specified by effective address.

$$D_4 T_4 : PC \leftarrow AR, SC \leftarrow 0$$

6. BSA (Branch & save return address)

- This instruction is useful for branching to a portion of the program called a subroutine/procedure.
- BSA stores the address of the next instruction in sequence available in PC into a memory location specified by effective address.
- Effective address plus 1 is then transferred

to PC to serve as address of the first instruction in subroutine.

$M[AR] \leftarrow PC, PC \leftarrow AR + 1$

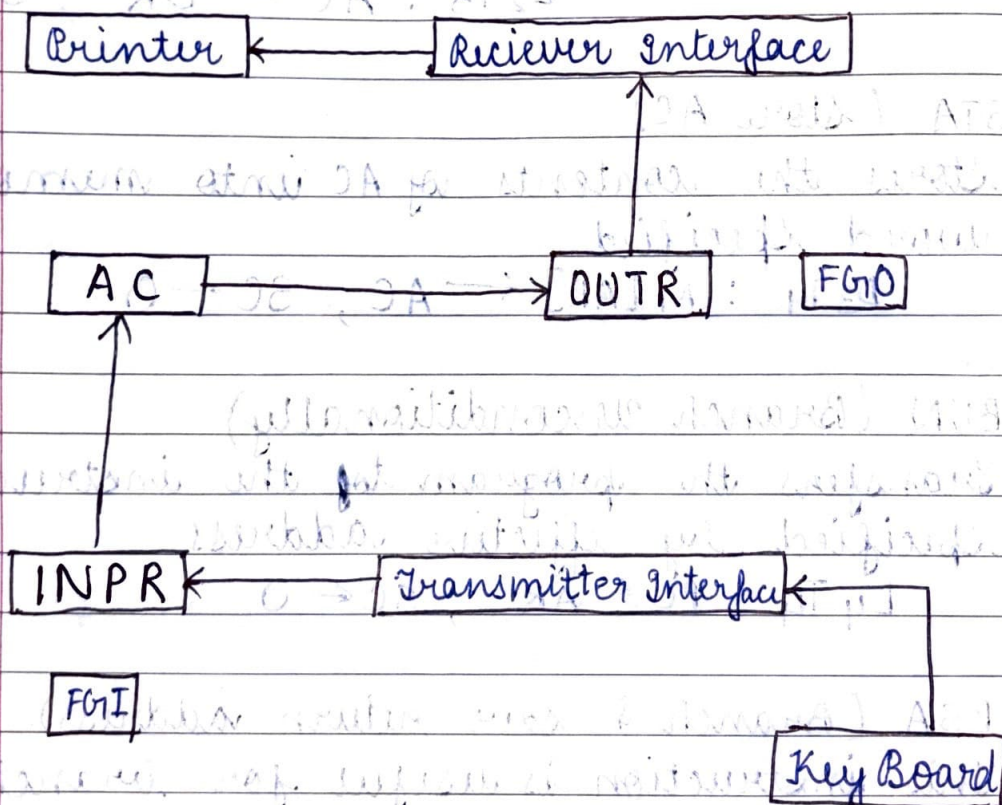
7. ISZ (Increment and skip if zero)

D6T4: $DR \leftarrow M[AR]$

D6T5: $DR \leftarrow DR + 1$

D6T6: $M[AR] \leftarrow DR$, if $(DR = 0)$ then $(PC \leftarrow PC + 1)$
 $(ISC \leftarrow 0)$

→ Input - output configuration



- 1 bit FG1 (flag input) is a control flip flop set to 1 when new information is available in the input device & is cleared to 0 when information is accepted by computer.

- Initially the output flag (FGO) is set to 1. The computer checks the flag bit, if it's 1, information from AC is transferred in parallel to OUTF and FGO is cleared to 0.
- Output device (printer) accepts this info and prints it and FGO is again set to 1.
- If $FGO = 0$, it means computer is printing a character.

→ Interrupt

- Computer checks again & again if the flag value has changed or not while processing several instructions and reducing the speed.
- To cure this, we let the external device inform the computer when it's ready for transfer & in meantime computer can carry on with other tasks.
- This type of transfer uses interrupt facility, if the flag is set, computer is momentarily interrupted from task it's doing & informed that flag is set.
- An interrupt enable flip flop can be used to do so.

$IEN = 0$ (can't interrupt)

$IEN = 1$ (can interrupt)

→ Reduced Instruction Set Computer (RISC) characteristics

Instructions ↓ Workdone ↑

1. Relatively few instructions
2. Relatively few addressing modes
3. Memory access is limited to load and store instructions
4. All operations are done within the registers of CPU
5. Fixed length easily decoded format
6. Single cycle instruction execution
7. Hardwired rather than microprogrammed control

→ Complex Instruction Set Computer (CISC) characteristics

1. A large no. of instructions
2. Some instructions perform specialised task & used infrequently
3. Large variety of addressing modes
4. Variable length instruction formats
5. Instructions that manipulate the operands in memory